

## PERTEMUAN 3

### POLIGON DAN ALGORITMA PENGISIAN POLIGON

#### Kemampuan Akhir yang Diharapkan (KAD):

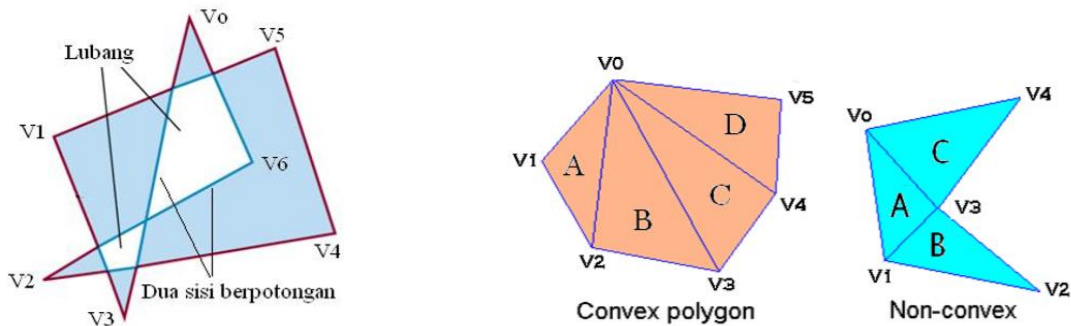
Mahasiswa mampu memahami dan mengimplementasikan algoritma pengisian polygon ke dalam program.

#### Pokok Bahasan:

1. Pengertian objek poligon
2. Algoritma *area filling scan line*
3. Algoritma *boundary-fill algorithm*
4. Algoritma *flood-fill algorithm*
5. Mengaplikasikan algoritma *area filling scan line*, *boundary-fill algorithm* dan *flood-fill algorithm* objek poligon ke dalam program komputer
6. Soal Latihan

#### 3.1 Pengertian Objek Poligon

Poligon adalah kumpulan garis lurus yang saling menyambung hingga membentuk suatu luasan. Garis-garis ini disebut *edge* (sisi poligon). Titik pertemuan tiap dua sisi disebut *verteks*. Biasanya poligon dinyatakan dengan koordinat verteks-verteks ini. Ada dua jenis poligon yaitu *poligon sederhana* dan *poligon tidak sederhana*. Ciri poligon sederhana adalah tidak semua verteks berada pada bidang yang sama, tidak mempunyai sisi yang berpotongan, dan tidak mempunyai lubang.



(a) Poligon tidak sederhana

(b) Poligon sederhana

Gambar 3.1 Poligon tidak sederhana dan polygon sederhana

Poligon sederhana dibagi menjadi dua yaitu *convex polygon* dan *non convex polygon*. Ciri *convex polygon* adalah *semua sudut interiornya  $< 180^\circ$* , atau setiap segmen garis yang dihasilkan dari dua buah verteks sembarang dalam poligon berada didalam poligon. Bentuk poligon yang paling sederhana adalah *segitiga*, karena semua poligon dapat dipecah-pecah menjadi bagian yang terkecil yaitu segitiga seperti pada Gambar 3.1(b). **Mengapa poligon?** Dengan poligon secara praktek kita bisa melakukan pendekatan untuk *membentuk permukaan setiap obyek 3-D*, jika kita mempunyai jumlah poligon yang cukup. Sebagai contoh permukaan bola, torus, dan teko seperti pada Gambar 3.2 berikut ini dapat dibuat dari beberapa poligon.



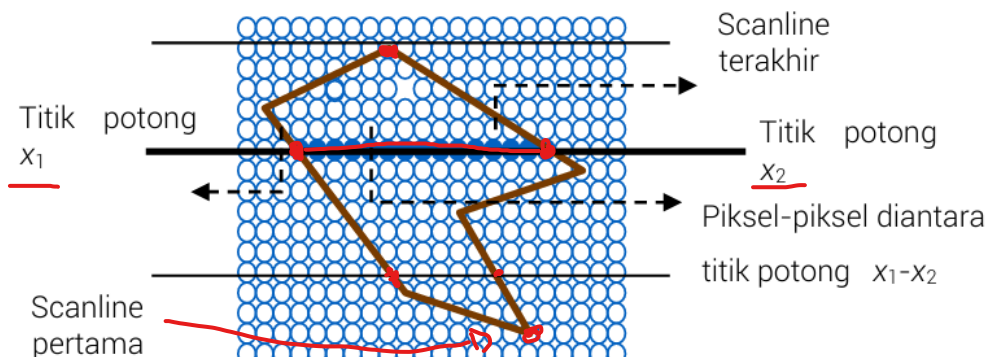
Gambar 3.2 Permukaan bola, torus dan teko yang terbentuk dari banyak poligon

### 3.2 Filling Polygon

Dalam grafis, pemberian warna dibutuhkan untuk mempercantik tampilan polygon. Karena itu diperlukan algoritma khusus untuk mengisi warna pada polygon tersebut. Teknik atau algoritma untuk pengisian warna pada polygon disebut *filling polygon* atau *area filling*. Ada dua macam dasar pendekatan *filling polygon* pada sistem raster yaitu *scan-line Polygon Fill Algorithm* dan *Boundary-Fill Algorithm*.

#### 3.2.1 Scan Line Polygon Fill Algorithms

Pemberian warna pada polygon dilakukan dengan *scan* secara horisontal dari kiri ke kanan. Hal ini dilakukan untuk mendapatkan titik potong dengan tepi polygon, kemudian mengurutkan nilai-nilai titik potong  $x$  dari kiri ke kanan dan memberi warna pada piksel-piksel di antara dua pasangan berurutan ( $x_1 - x_2$ ). Hal ini dilakukan dari garis scan yang paling bawah (nilai  $y$  terkecil) hingga garis scan yang paling atas seperti pada Gambar 3.3. Metode ini bisa juga digunakan untuk pengisian warna pada obyek-obyek sederhana lainnya, misalnya lingkaran, ellip dan lain-lain.

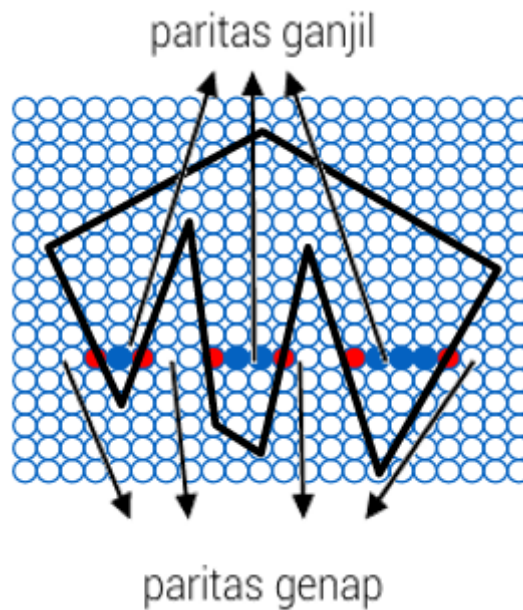


Gambar 3.3 Metode Scan-Line

Bagaimana kita tahu bahwa piksel tersebut berada didalam polygon atau diluar polygon?

#### A. Inside-Outside test

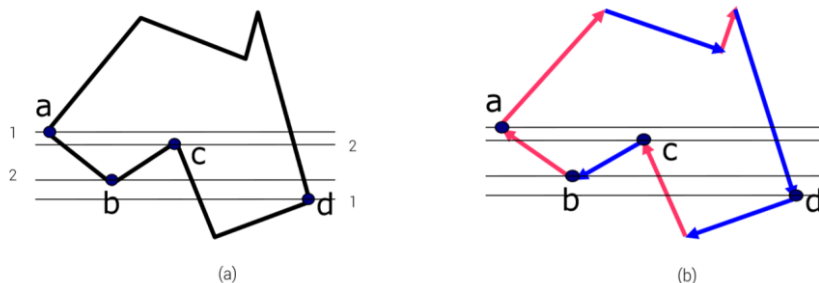
Perhatikan Gambar 3.4. Untuk mengidentifikasi bagian dalam dan bagian luar digunakan aturan *paritas ganjil-genap* yang biasa disebut sebagai *odd-even rule* atau *odd parity rule*. Semula kita set paritas dengan *nilai genap*. Setiap ditemukan titik potong nilai paritas dibalik, yang semula *genap* dibalik menjadi *ganjil*, sebaliknya yang semula *ganjil* dibalik menjadi *genap*. Beri warna pada piksel jika paritasnya *Ganjil*.



Gambar 3.4 Aturan paritas ganjil-genap untuk mengisi warna

**Masalah:**

Pada Gambar 3.5(a), verteks a, b, c, dan d merupakan pertemuan dari dua buah segmen garis. Mengapa pada verteks a dan d dihitung sekali, sedangkan pada verteks b dan c dihitung dua kali?

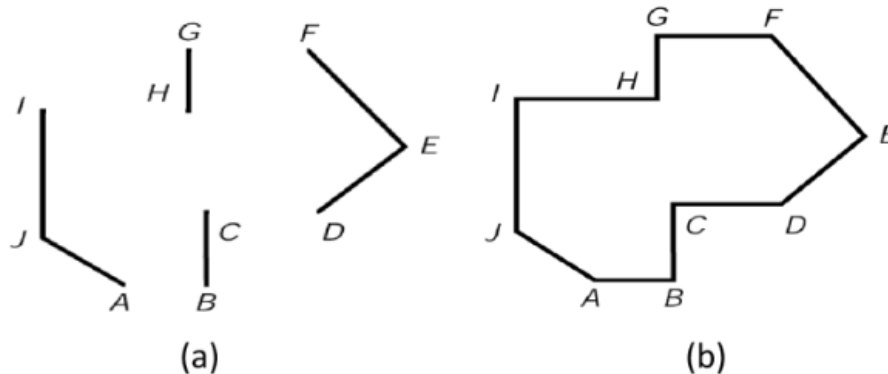


Gambar 3.5 Perhitungan paritas pada verteks a, b, c, dan d

**Solusi:** Buatlah perputaran tepi polygon searah jarum jam seperti pada Gambar 3.5(b). *Check*, apabila pada suatu verteks arah anak panahnya berubah (pada verteks b dan c yaitu: naik-turun atau turun-naik) maka pada verteks tersebut dihitung dua kali. Sebaliknya jika arah anak panahnya tidak berubah (pada verteks a dan d yaitu: naik-naik atau turun-turun) maka pada verteks tersebut dihitung sekali.

**Masalah:**

Perhatikan **Gambar 4.6(a)**, Bagaimana perhitungan tepi horisontal **AB**, **CD**, **GF**, dan **HI**?



Gambar 3.6 Polygon dengan sisi-sisi horisontal

**Solusi:** Abaikan verteks-verteks yang terletak pada tepi horisontal, atau jangan dimasukkan dalam perhitungan paritas ganjil-genap, sehingga tampak seperti pada **Gambar 3.6(b)**.

Algoritma scan-line menggunakan kaidah paritas ganjil-genap:

- Tentukan titik potong garis scan dengan semua sisi polygon.
- Urutkan titik potong tadi berdasarkan koordinat sumbu x.
- Warnai semua piksel diantara pasangan titik potong ( $x_1-x_2$ ) yang terletak di dalam polygon menggunakan kaidah paritas ganjil-genap.

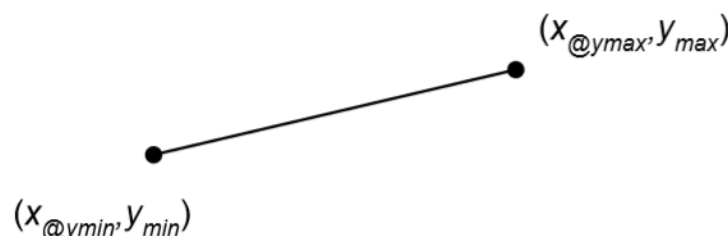
Kelemahan algoritma ini adalah:

Memerlukan biaya tinggi (*big cost*) karena pada setiap sisi polygon selalu dilakukan pengujian terhadap piksel-piksel.

**Solusi:** Menggunakan konsep *edge table* (ET)

### B. Edge Table (ET)

*Edge Table* (ET) adalah tabel yang berisi sisi-sisi polygon. Kita akan menggunakan dua *edge table* yang berbeda, yaitu: *Active Edge Table* (AET) dan *Global Edge Table* (GET). AET digunakan untuk menyimpan semua sisi polygon yang berpotongan dengan garis *scan*, sedangkan GET digunakan untuk menyimpan semua sisi polygon dan meng-update AET. Perhitungan titik potong dengan garis *scan*:



Kita tahu bahwa:

$$y = mx + b, \quad m = \frac{y_{\max} - y_{\min}}{x_{@y\max} - x_{@y\min}}$$

Setiap garis *scan* yang baru berlaku :

$$y_{i+1} = y_i + 1$$

Kita perlu menghitung  $x$  untuk setiap garis scan:

$$x = \frac{y-b}{m}$$

Sehingga,

$$x_i = \frac{y_i - b}{m}, \quad x_{i+1} = \frac{y_{i+1} - b}{m}$$

Dan

$$y_{i+1} = y_i + 1$$

Maka:

$$x_{i+1} = \frac{y_i + 1 - b}{m} = \frac{y_i - b}{m} + \frac{1}{m} = x_i + \frac{1}{m}$$

Ini adalah cara yang efisien untuk menghitung nilai  $x$ .

### C. Active Edge Table (AET)

- Tabel berisi satu *entry* per sisi yang berpotongan dengan garis scan aktif.

- $x_{i+1} = x_i + \frac{1}{m}$  Pada setiap garis scan yang baru :
  - Hitung titik potong baru untuk semua sisi menggunakan rumus :
  - Tambahkan setiap sisi baru yang berpotongan
  - Hapus setiap sisi yang tidak berpotongan
- Untuk efisiensi Update AET, kita tetap menjaga GET.

### D. Global Edge Table (GET)

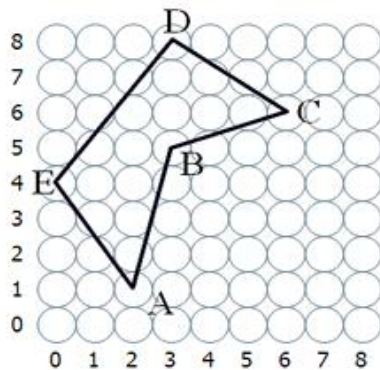
- Tabel yang berisi informasi tentang semua sisi-sisi polygon.
- GET mempunyai satu tempat untuk setiap garis scan.
  - Setiap tempat menyimpan daftar sisi yang mempunyai nilai  $y_{\min}$
  - Setiap sisi ditentukan hanya dalam satu tempat
- Tiap-tiap entry dalam GET berisi
  - Nilai  $y_{\max}$  dari sisi
  - Nilai  $x_{@y_{\min}}$  (nilai  $x$  pada titik  $y_{\min}$ )
  - Nilai pertambahan  $x$  ( $1/m$ )

### E. Scan Line Polygon Fill Algorithms menggunakan ET, GET dan AET

1. Tambahkan sisi-sisi polygon ke GET
2. Set  $y$  ke koordinat  $y$  terkecil dalam GET
3. Inisialisasi, set AET = kosong
4. Ulang sampai AET dan GET kosong
  - a. Tambahkan sisi-sisi dari GET ke AET yang mana  $y_{\min} = y$ .
  - b. Hilangkan sisi-sisi dari AET bila  $y_{\max} = y$ .
  - c. Urutkan AET berdasarkan  $x$
  - d. Warnai piksel yang terletak diantara pasangan titik potong dalam AET
  - e. Untuk setiap sisi dalam AET, ganti  $x$  dengan  $x + 1/m$
  - f. Set  $y = y + 1$  untuk bergerak ke garis scan berikutnya.

**Contoh soal:**

Diketahui polygon berikut, gunakan konsep *edge table* untuk mewarnai polygon tersebut!



Masukan GET ( $y_{\max}$ ,  $x_{@y_{\min}}$ ,  $1/m$ )

AB  $\rightarrow (5, 2, \frac{1}{4})$

BC  $\rightarrow (6, 3, 3)$

CD  $\rightarrow (8, 6, -\frac{3}{2})$

DE  $\rightarrow (8, 0, \frac{3}{4})$

EA  $\rightarrow (4, 2, -\frac{2}{3})$

Sisi-sisi pembentuk polygon:

AB = (2,1), (3,5)

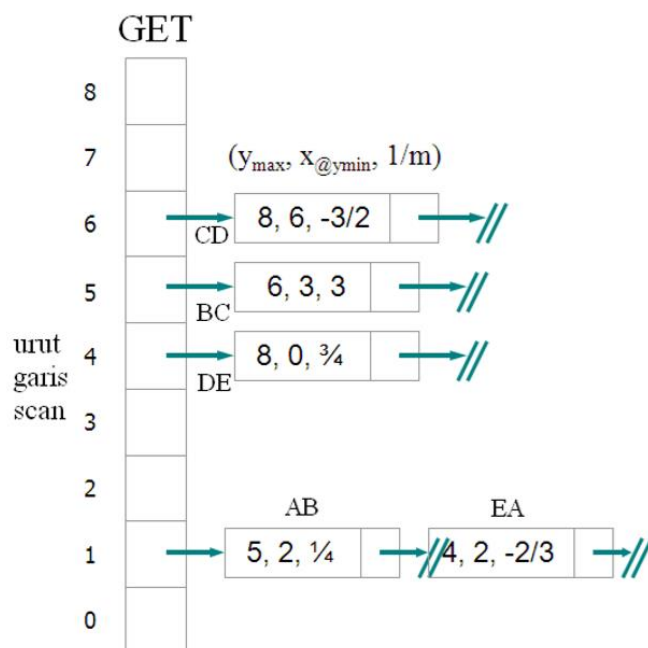
BC = (3,5), (6,6)

CD = (6,6), (3,8)

DE = (3,8), (0,4)

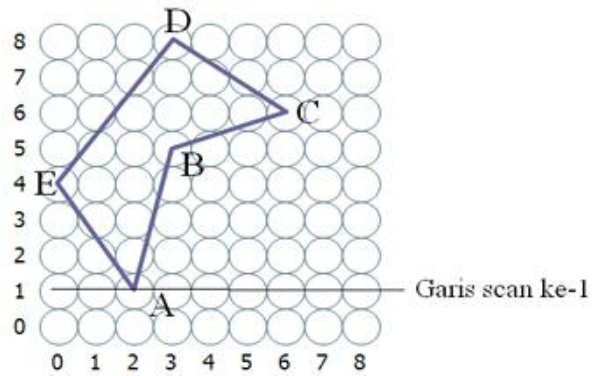
EA = (0,4), (2,1)

Tempatkan masukan ke dalam GET berdasarkan nilai  $y_{\min}$ .

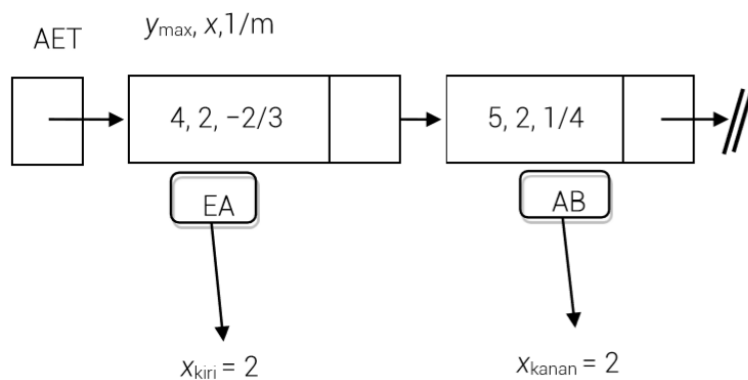


Gambar 3.7 GET berdasarkan nilai  $y_{\min}$

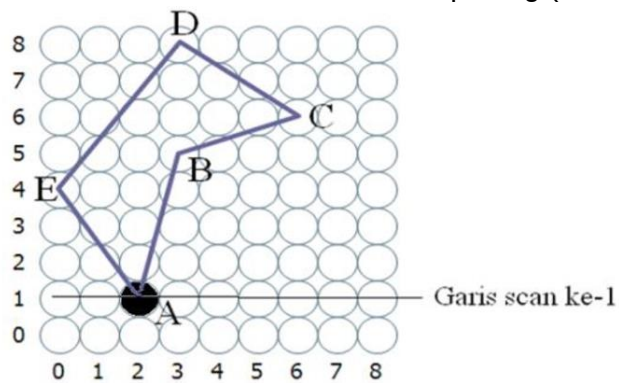




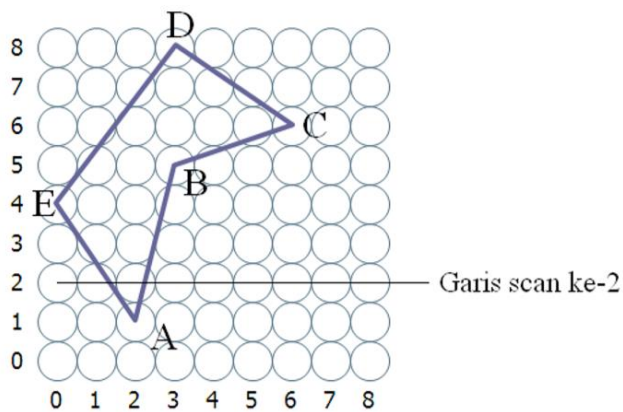
Gambar 3.8 Scan Line ke-1



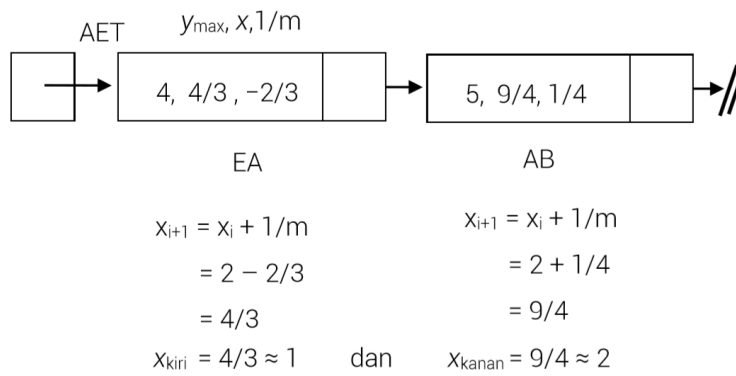
Pewarnaan dilakukan diantara titik potong ( $x_{\text{kiri}} - x_{\text{kanan}} = (2 - 2)$ ), hasilnya adalah:



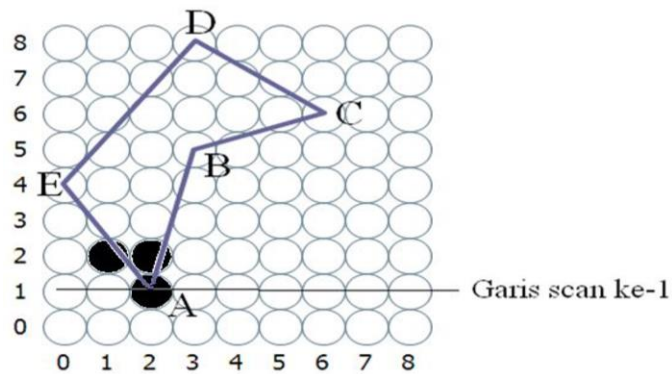
Gambar 3.9 Hasil Scan Line ke-1



Gambar 3.10 Scan Line ke-2

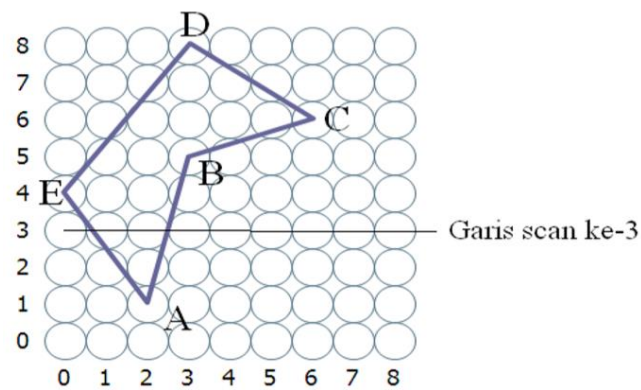


Pewarnaan dilakukan diantara titik potong ( $x_{\text{kiri}} - x_{\text{kanan}} = (1 - 2)$ ), hasilnya adalah:

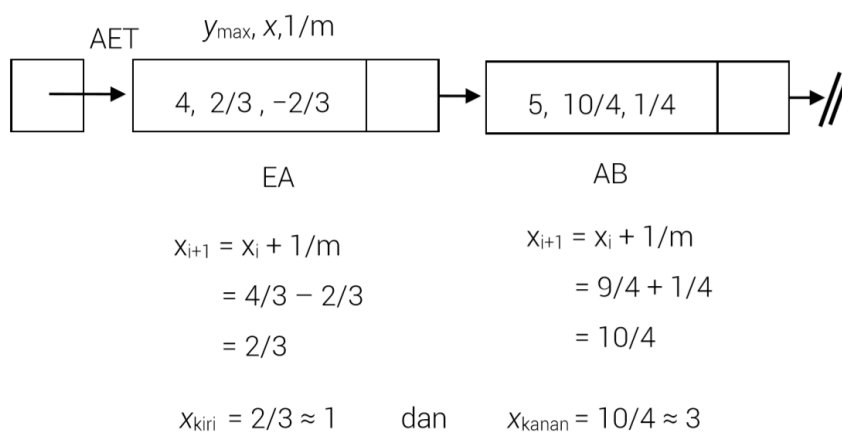


Gambar 3.11 Hasil Scan Line ke-

2

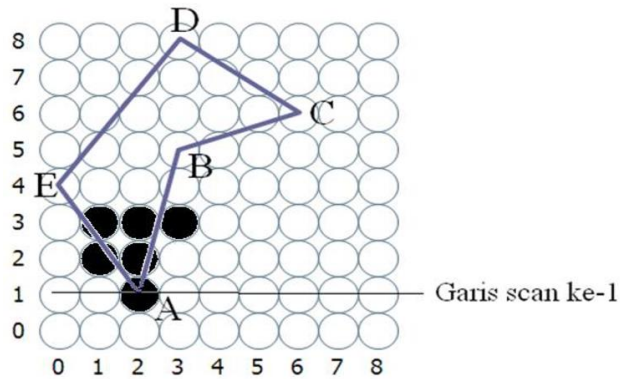


Gambar 3.12 Scan Line ke-3

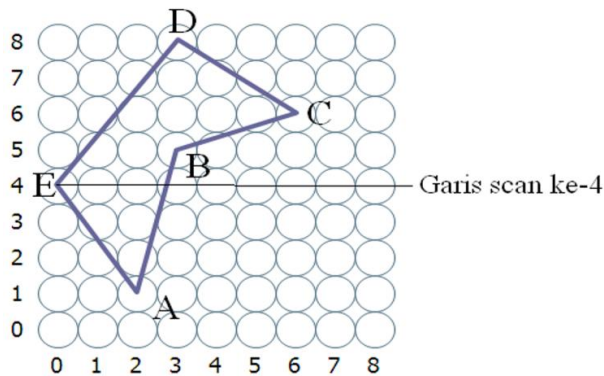




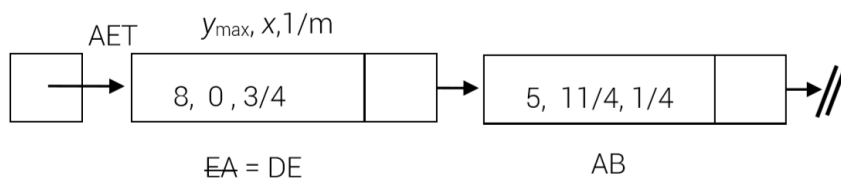
Pewarnaan dilakukan diantara titik potong ( $x_{kiri} - x_{kanan}$ ) = (1 - 3), hasilnya adalah:



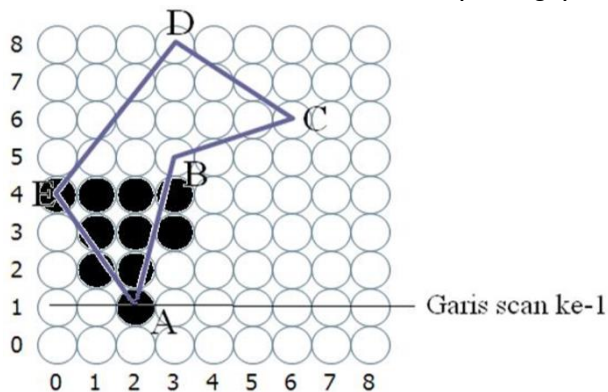
Gambar 3.13 Hasil Scan Line ke-3



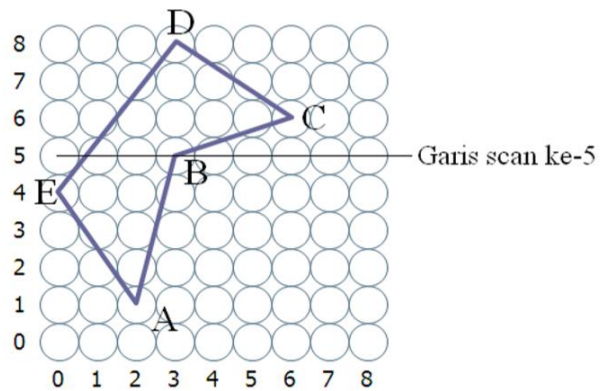
Gambar 3.14 Scan Line ke 4



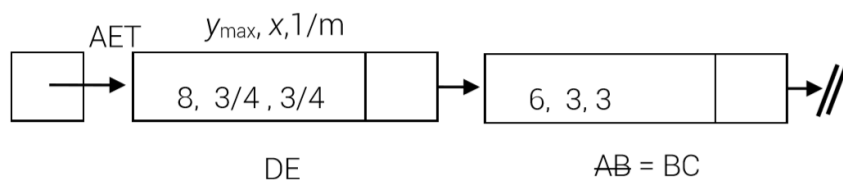
Pewarnaan dilakukan diantara titik potong ( $x_{kiri} - x_{kanan}$ ) = (0 - 3), hasilnya adalah:



Gambar 3.15 Hasil Scan Line ke-4



Gambar 3.16 Scan Line ke-5



$$\begin{aligned}
 x_{i+1} &= x_i + 1/m \\
 &= 0 + 3/4 \\
 &= 3/4
 \end{aligned}$$

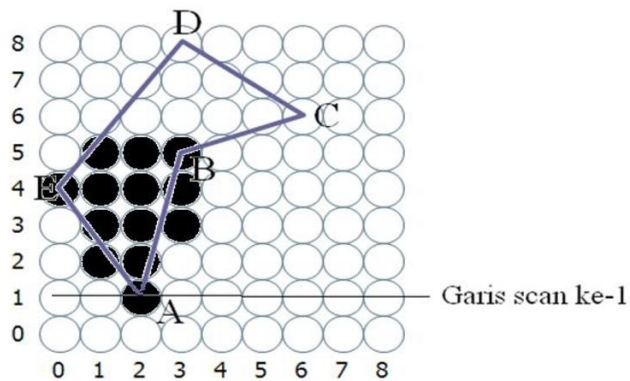
pada AB,  $y_{\max} = 5$ . AB harus dihapus dari AET. Dalam GET sisi BC tersimpan pada  $y = 5$ . Jadi sisi AB diganti dengan sisi BC

$$x_{\text{kiri}} = 3/4 \approx 1$$

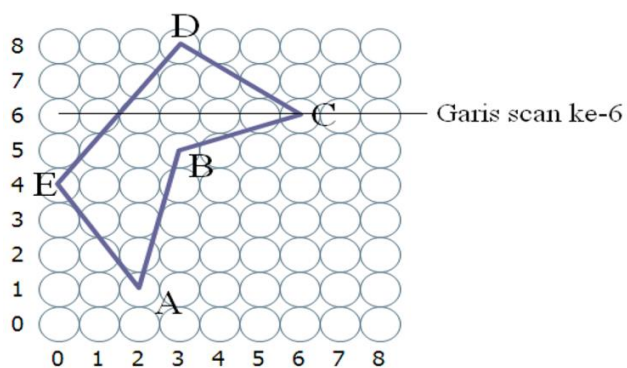
dan

$$x_{\text{kanan}} = 3$$

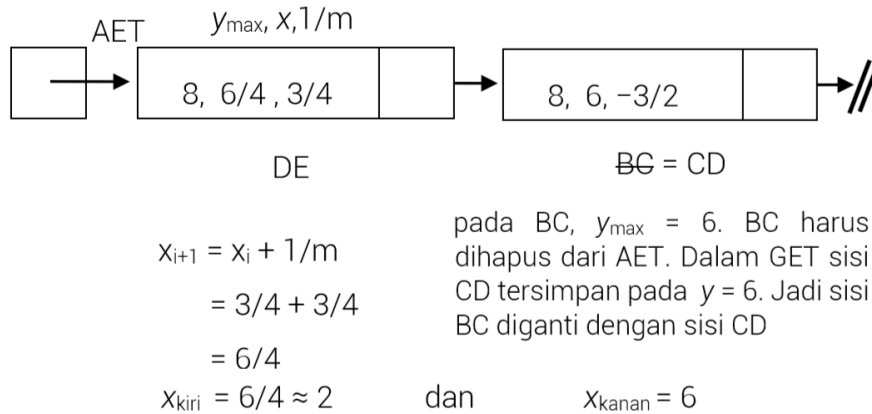
Pewarnaan dilakukan diantara titik potong ( $x_{\text{kiri}} - x_{\text{kanan}} = (1 - 3)$ ), hasilnya adalah:



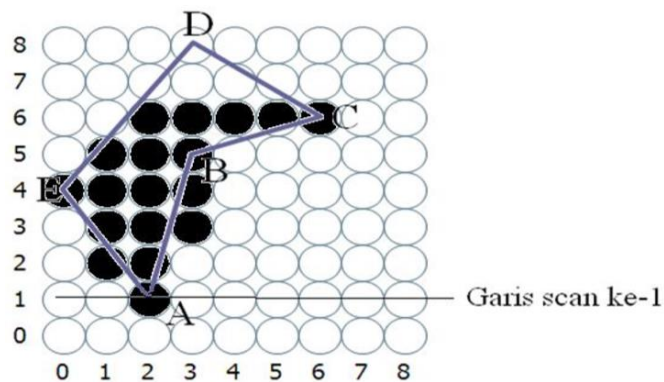
Gambar 3.17 Hasil Scan Line ke-5



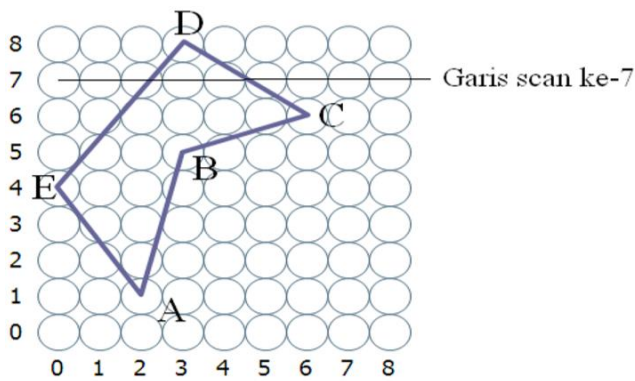
Gambar 3.18 Scan Line ke-6



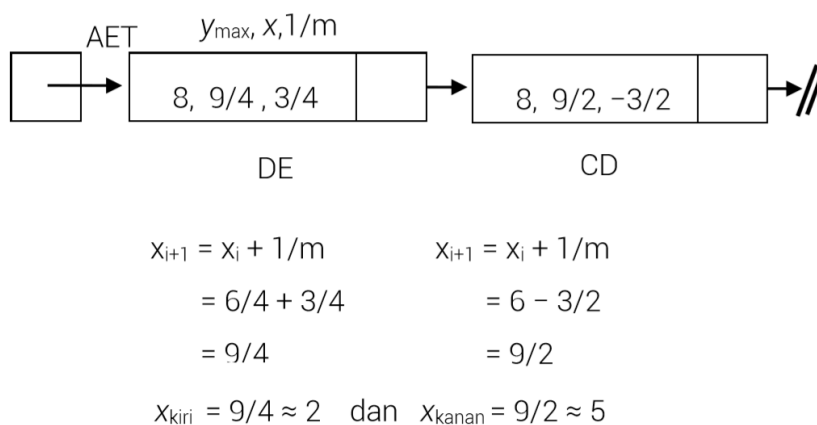
Pewarnaan dilakukan diantara titik potong ( $x_{\text{kiri}} - x_{\text{kanan}} = (2 - 6)$ ), hasilnya adalah:



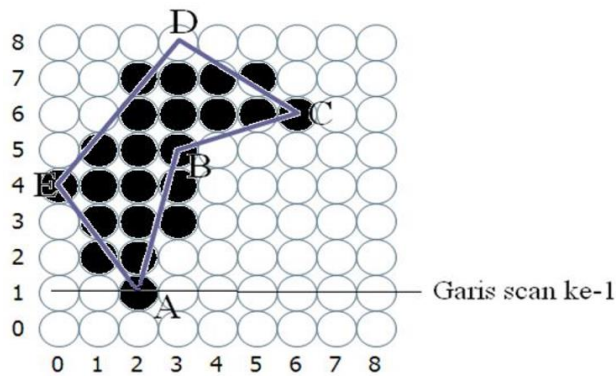
Gambar 3.19 Hasil Scan Line ke-6



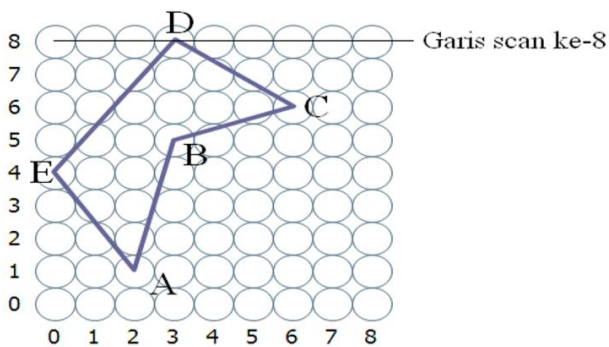
Gambar 3.20 Scan Line ke-7



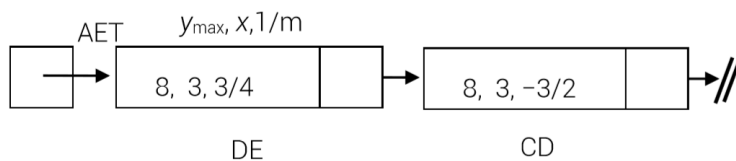
Pewarnaan dilakukan diantara titik potong ( $x_{kiri} - x_{kanan}$ ) = (2 - 5), hasilnya adalah:



Gambar 3.21 Hasil Scan Line ke-7

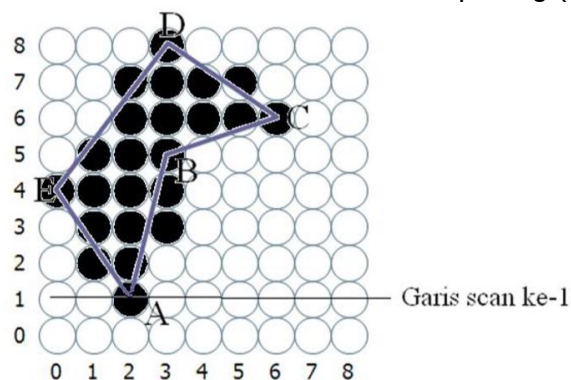


Gambar 3.22 Scan Line ke-8



$$\begin{aligned}
 x_{i+1} &= x_i + 1/m & x_{i+1} &= x_i + 1/m \\
 &= 9/4 + 3/4 & &= 9/2 - 3/2 \\
 &= 3 & &= 3 \\
 x_{kiri} &= 3 & \text{dan} & x_{kanan} = 3
 \end{aligned}$$

Pewarnaan dilakukan diantara titik potong ( $x_{kiri} - x_{kanan}$ ) = (3 - 3), hasilnya adalah:



Gambar 3.23 Hasil Scan Line ke-8

Karena sisi polygon dalam GET ataupun AET sudah habis, maka proses dihentikan.

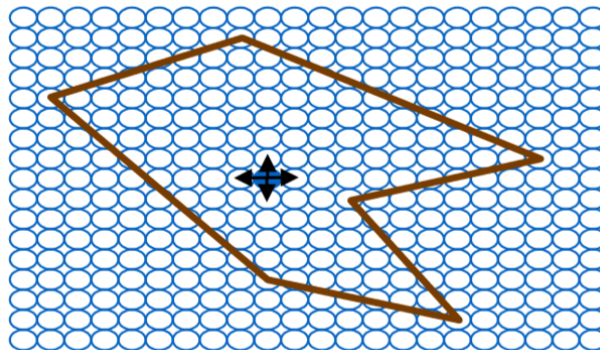
### 3.2.2 Boundary-Fill Algorithm

Algoritma *Boundary Fill* bekerja sesuai namanya. Algoritma ini mengambil titik dalam sebuah obyek dan mulai mengisi sampai menyentuh batas objek. Warna batas dan warna yang kita isi harus berbeda untuk algoritma ini bekerja. *Boundary Fill* merupakan salah satu pendekatan untuk *area filling pada output primitive* dengan memulai dari suatu titik didalam region dan mewarnai interior keluar batas. Jika *boundary* dispesifikasikan dengan satu warna, *fill algorithm* diproses per piksel sampai warna *boundary* didapat. Algoritma ini menerima input koordinat titik interior (x,y), warna isi dan warna *boundary*. Mulai dari titik tersebut, prosedur akan menguji posisi *connected* untuk menentukan apakah posisi tersebut ada di *boundary color*.

Pendekatan ini bermanfaat untuk paket aplikasi grafik interaktif, dimana titik dalam dapat dengan mudah ditentukan. Prosedurnya yaitu menerima input koordinat dari suatu titik (x,y), warna isi dan warna garis batas. Proses pengisian warna tertentu dimulai dari titik (x,y), kemudian memeriksa posisi titik *connected*-nya, apakah titik *connected* tersebut memiliki warna batas:

- Jika tidak, warnai titik tersebut dengan warna tertentu.
- Selanjutnya periksa lagi posisi dan warna titik *connected*-nya.
- Proses diulangi terus hingga seluruh titik pada area pengisian telah diuji.

Dengan teknik ini pengisian warna dimulai dari sebuah titik yang berada didalam area (x,y) polygon dan mewarnai piksel dari titik *connected* nya hingga semua piksel yang berada didalam polygon telah diwarnai seperti pada Gambar 3.24.



Gambar 3.24 Metode Boundary-Fill

Berikut adalah prosedur algoritma *Boundary-Fill*:

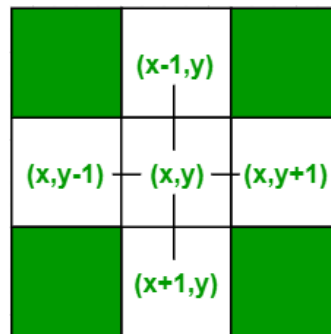
```
void BoundaryFill(int x, int y, int fill, int boundary)
{
    int current;
    current = getPixel(x, y);
    if ((current != boundary) && (current != fill))
    {
        setColor(fill);
        setPixel(x, y);
        BoundaryFill(x + 1, y, fill, boundary);
        BoundaryFill(x - 1, y, fill, boundary);
        BoundaryFill(x, y + 1, fill, boundary);
        BoundaryFill(x, y - 1, fill, boundary);
    }
}
```



Ada 2 macam cara untuk melihat titik *connected*, yaitu :

- **4 *connected***, melihat titik yang berada diatas, bawah, kanan dan kiri
- **8 *connected***, melihat titik yang berada diatas, bawah, kanan, kiri, pojok kiri atas, pojok kiri bawah, pojok kanan atas dan pojok kanan bawah.

### 1. Metode 4-connected



Gambar 3.25 Metode 4-connected

Dalam teknik ini 4 *connected* digunakan seperti yang ditunjukkan pada gambar. Kita menempatkan piksel atas, di bawah, ke kanan, dan ke sisi kiri piksel dan proses ini akan berlanjut sampai kita menemukan batas dengan warna yang berbeda.

#### Algoritma:

Langkah 1 - Inisialisasi nilai titik tengah (seedx, kumuh), fcolor dan dcolor

Langkah 2 - Menentukan nilai batas poligon

Langkah 3 - Periksa apakah titik tengah saat ini adalah warna default, kemudian ulangi langkah 4 dan 5 sampai piksel batas tercapai.

```
If getpixel(x, y) = dcol then repeat step 4 and 5
```

Langkah 4 - Mengubah warna *default* dengan warna isi pada titik berikutnya.

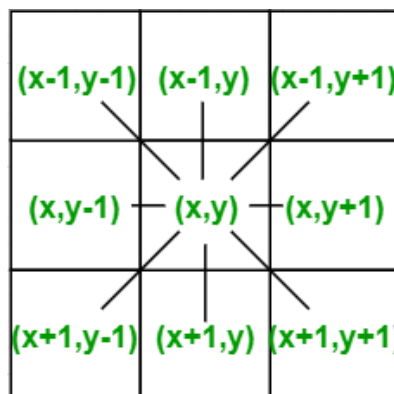
```
setPixel(seedx, seedy, fcol)
```

Langkah 5 - Secara rekursif ikuti prosedur dengan empat poin titik sekitarnya.

```
BoundaryFill (seedx - 1, seedy, fcol, dcol)
BoundaryFill (seedx + 1, seedy, fcol, dcol)
BoundaryFill (seedx, seedy - 1, fcol, dcol)
BoundaryFill (seedx, seedy + 1, fcol, dcol)
```

Langkah 6 – Selesai.

### 2. Metode 8-connected



Gambar 3.26 Metode 8-connected



Dalam teknik ini 8-connected digunakan seperti yang ditunjukkan pada gambar. Kita menempatkan piksel di atas, bawah, sisi kanan dan kiri dari pixel saat seperti yang kami lakukan dalam teknik 4-connected dan juga pada bagian diagonal-diagonalnya.

**Algoritma:**

Langkah 1 - Inisialisasi nilai titik tengah (seedx , seedy) , fcolor dan dcolor.

Langkah 2 - Menentukan nilai batas poligon

Langkah 3 - Periksa apakah titik tengah saat ini adalah warna default, kemudian ulangi langkah 4 dan 5 sampai piksel batas tercapai.

```
If getpixel(x, y) = dcol then repeat step 4 and 5
```

Langkah 4 - Mengubah warna default dengan warna isi pada titik berikutnya.

```
setPixel(seedx, seedy, fcol)
```

Langkah 5 - Secara rekursif ikuti prosedur dengan empat poin titik sekitarnya.

```
BoundaryFill (seedx - 1, seedy, fcol, dcol)
BoundaryFill (seedx + 1, seedy, fcol, dcol)
BoundaryFill (seedx, seedy - 1, fcol, dcol)
BoundaryFill (seedx, seedy + 1, fcol, dcol)
BoundaryFill (seedx - 1, seedy + 1, fcol, dcol)
BoundaryFill (seedx + 1, seedy + 1, fcol, dcol)
BoundaryFill (seedx + 1, seedy - 1, fcol, dcol)
BoundaryFill (seedx - 1, seedy - 1, fcol, dcol)
```

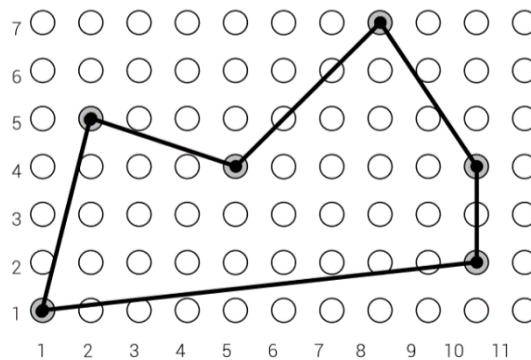
Langkah 6 – Selesai.

Contoh:

Diketahui : polygon = {(1,1), (2,5), (5,4), (8,7), (10,4), (10,2), (1,1)}, lakukan Area Filling menggunakan algoritma Boundary Fill Algorithm 4-connected dan 8-connected !

Penyelesaian:

titik-titik sebagai pembentuk polygon = {(1,1), (2,5), (5,4), (8,7), (10,4), (10,2), (1,1)}. Bila poligon tersebut digambar, diperoleh gambar berikut:

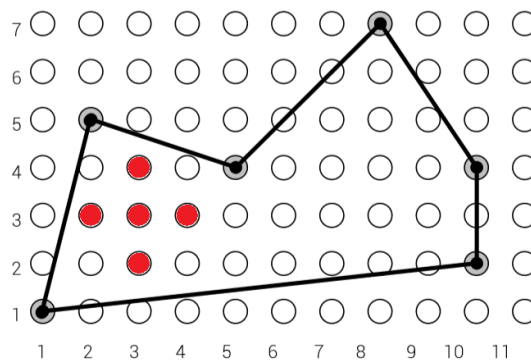


Gambar 3.27 Polygon soal

**a. Analisa Metode 4-connected**

Langkah 1:

Misalkan titik awal pencarian adalah (3,3). Tandai titik (3,3) dengan warna tertentu, misalnya warna merah. Kemudian menentukan titik tetangga yaitu titik (2,3), (3,2), (3,4), dan (4,3). Karena titik-titik ini bukan batas poligon maka mereka juga diwarnai.



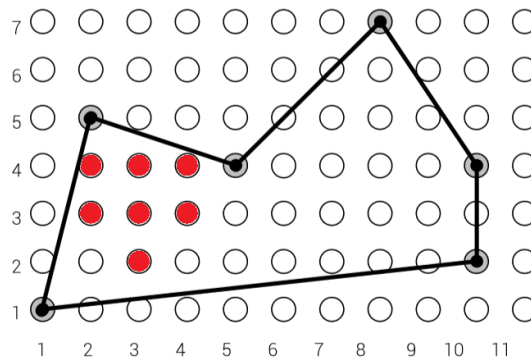
Gambar 3.28 Langkah 1

Titik yang telah diproses: (3,3)

Titik yang belum diproses : (3,2), (3,4), (2,3), (4,3)

Langkah 2:

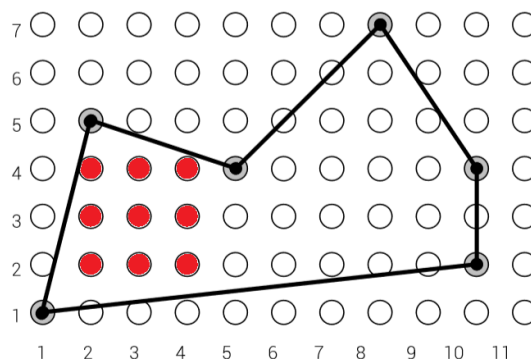
Kemudian diambil titik (3,4) sebagai titik tengah dan titik tetangganya yaitu (3,3), (2,4), (4,4) dan (3,5). Titik (3,3) telah diproses. Titik (2,4) dan (4,4) belum diproses dan masih dalam batas sehingga diberi warna yaitu merah, sedangkan titik (3,5) berada diluar batas sehingga tidak diwarnai.



Gambar 3.29 Langkah 2

Langkah 3:

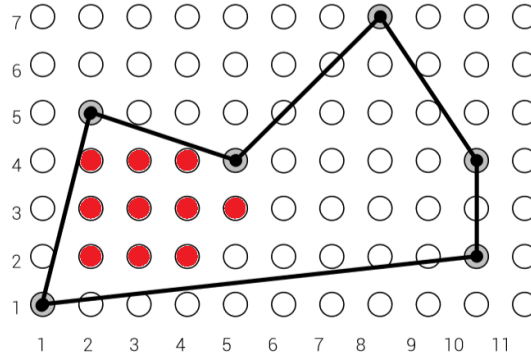
Kemudian diambil titik (3,2) sebagai titik tengah dan titik tetangganya yaitu (3,3), (2,2), (4,2) dan (3,1). Titik (3,3) telah diproses. Titik (2,2) dan (4,2) belum diproses dan masih dalam batas sehingga diberi warna yaitu merah, sedangkan titik (3,1) berada diluar batas sehingga tidak diwarnai.



Gambar 3.30 Langkah 3

Langkah 4:

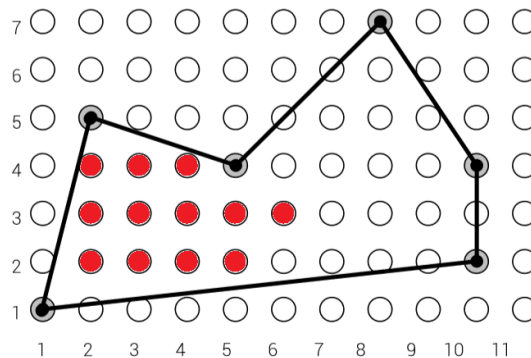
Kemudian diambil titik (4,3) sebagai titik tengah dan titik tetangganya yaitu (3,3), (4,2), (4,4) dan (5,3). Titik (3,3), (4,2), dan (4,4) telah diproses. Titik (5,3) belum diproses dan masih dalam batas sehingga diberi warna merah.



Gambar 3.31 Langkah 4

Langkah 5:

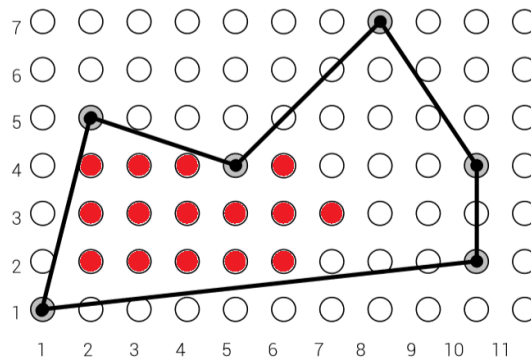
Kemudian diambil titik (5,3) sebagai titik tengah dan titik tetangganya yaitu (4,3), (5,2), (5,4) dan (6,3). Titik (4,3) telah diproses. Titik (5,2) dan (6,3) belum diproses dan masih dalam batas sehingga diberi warna merah. Sedangkan titik (5,4) merupakan titik batas sehingga tidak diberi warna.



Gambar 3.31 Langkah 5

Langkah 6:

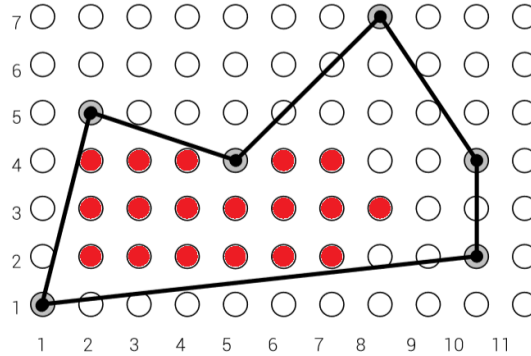
Kemudian diambil titik (6,3) sebagai titik tengah dan titik tetangganya yaitu (5,3), (6,2), (6,4) dan (7,3). Titik (5,3) telah diproses. Titik (6,2), (6,4) dan (7,3) belum diproses dan masih dalam batas sehingga diberi warna merah.



Gambar 3.32 Langkah 6

Langkah 7:

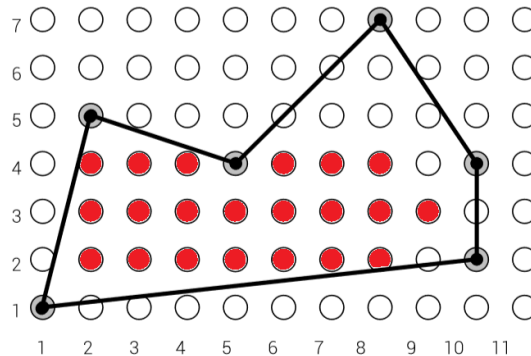
Kemudian diambil titik (7,3) sebagai titik tengah dan titik tetangganya yaitu (6,3), (7,2), (7,4) dan (8,3). Titik (6,3) telah diproses. Titik (7,2), (7,4) dan (8,3) belum diproses dan masih dalam batas sehingga diberi warna merah.



Gambar 3.33 Langkah 7

Langkah 8:

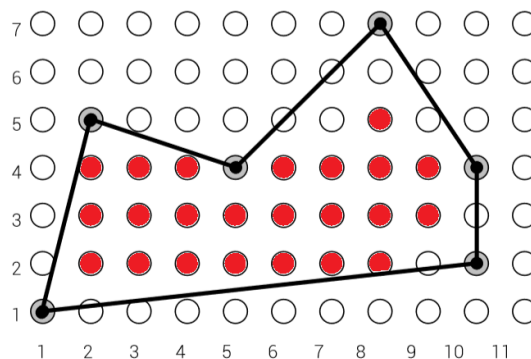
Kemudian diambil titik (8,3) sebagai titik tengah dan titik tetangganya yaitu (7,3), (8,2), (8,4) dan (9,3). Titik (7,3) telah diproses. Titik (8,2), (8,4) dan (9,3) belum diproses dan masih dalam batas sehingga diberi warna merah.



Gambar 3.34 Langkah 8

Langkah 9:

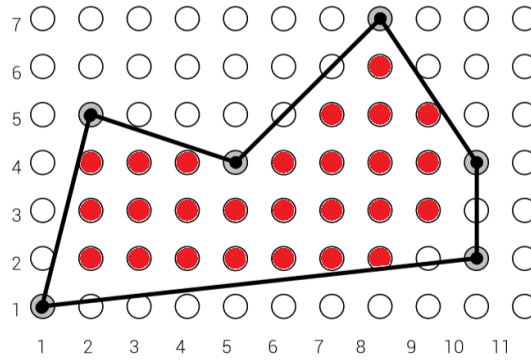
Kemudian diambil titik (8,4) sebagai titik tengah dan titik tetangganya yaitu (7,4), (8,3), (8,5) dan (9,4). Titik (7,4) dan (8,3) telah diproses. Titik (8,5) dan (9,4) belum diproses dan masih dalam batas sehingga diberi warna merah.



Gambar 3.35 Langkah 9

Langkah 10:

Kemudian diambil titik (8,5) sebagai titik tengah dan titik tetangganya yaitu (7,5), (8,4), (8,6) dan (9,5). Titik (8,4) telah diproses. Titik (7,5), (8,6) dan (9,5) belum diproses dan masih dalam batas sehingga diberi warna merah. Inilah hasil akhir dari pewarnaan dengan menggunakan metode 4-connected.

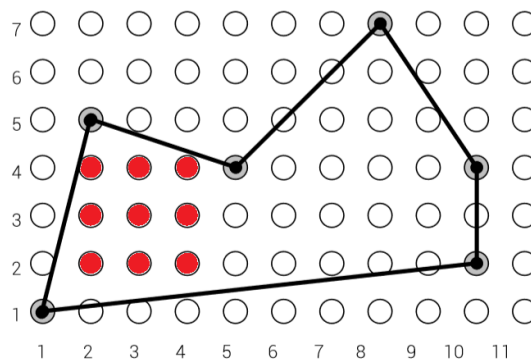


Gambar 3.36 Langkah 10

#### b. Analisa Metode 8-connected

Langkah 1:

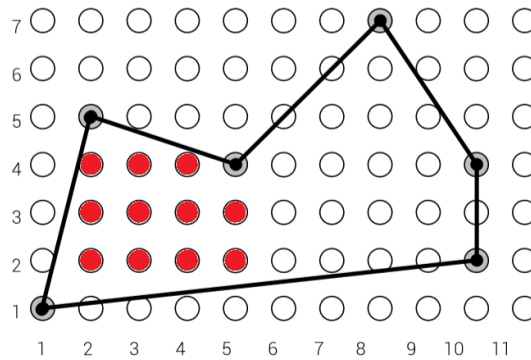
Diambil titik (3,3) sebagai titik tengah dan diberi warna merah, kemudian menentukan 8 titik tetangga yaitu titik (2,3), (2,4), (3,4), (4,4), (4,3), (4,2), (3,2) dan (2,2). Karena titik-titik ini bukan batas poligon maka mereka juga diwarnai.



Gambar 3.37 Langkah 1

Langkah 2:

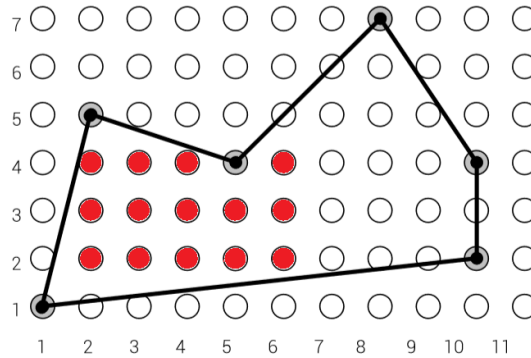
Kemudian diambil titik (4,3) sebagai titik tengah dan titik tetangganya yaitu (3,3), (3,4), (4,4), (5,4), (5,3), (5,2), (4,2), dan (3,2). Titik (4,2), (3,2), (3,3), (3,4) dan (4,4) telah diproses. Titik (5,3) dan (5,2) belum diproses dan masih dalam batas sehingga diberi warna merah, sedangkan titik (5,4) berada pada batas sehingga tidak diwarnai.



Gambar 3.38 Langkah 2

Langkah 3:

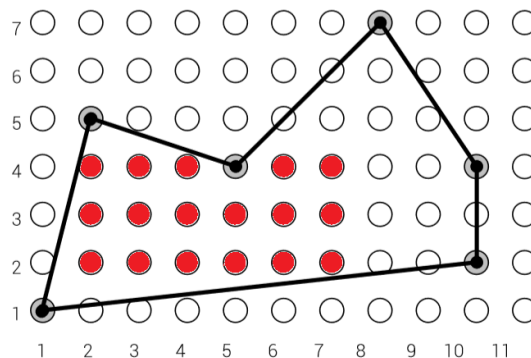
Kemudian diambil titik (5,3) sebagai titik tengah dan titik tetangganya yaitu (4,3), (4,4), (5,4), (6,4), (6,3), (6,2), (5,2), dan (4,2). Titik (5,2), (4,2), (4,3) dan (4,4) telah diproses. Titik (6,4), (6,3) dan (6,2) belum diproses dan masih dalam batas sehingga diberi warna merah, sedangkan titik (5,4) berada pada batas sehingga tidak diwarnai.



Gambar 3.39 Langkah 3

Langkah 4:

Kemudian diambil titik (6,3) sebagai titik tengah dan titik tetangganya yaitu (5,3), (5,4), (5,4), (7,4), (7,3), (7,2), (6,2), dan (5,2). Titik (6,2), (5,2), (5,3) dan (6,4) telah diproses. Titik (7,4), (7,3) dan (7,2) belum diproses dan masih dalam batas sehingga diberi warna merah, sedangkan titik (5,4) berada pada batas sehingga tidak diwarnai.

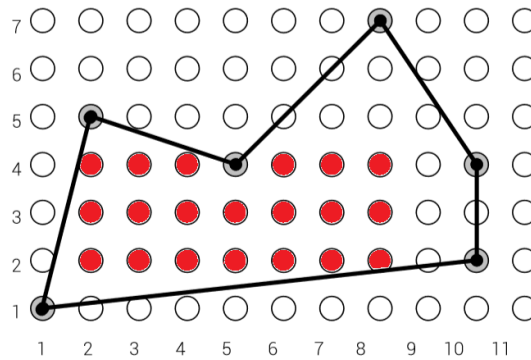


Gambar 3.40 Langkah 4

Langkah 5:

Kemudian diambil titik (7,3) sebagai titik tengah dan titik tetangganya yaitu (6,3), (6,4), (7,4), (8,4), (8,3), (8,2), (7,2), dan (6,2). Titik (7,2), (6,2), (6,3), (6,4) dan (7,4) telah diproses. Titik (8,4), (8,3) dan (8,2) belum diproses dan masih dalam batas sehingga diberi warna merah.

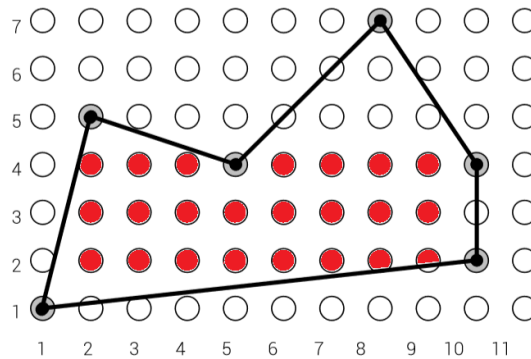




Gambar 3.41 Langkah 5

Langkah 6:

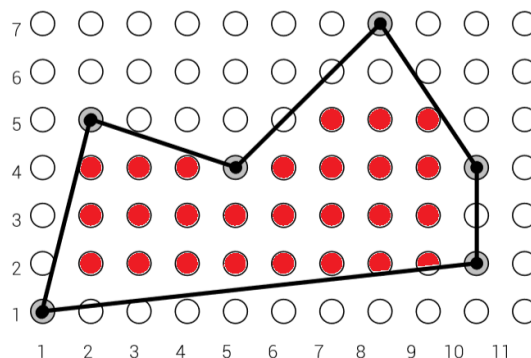
Kemudian diambil titik (8,3) sebagai titik tengah dan titik tetangganya yaitu (7,3), (7,4), (8,4), (9,4), (9,3), (9,2), (8,2), dan (7,2). Titik (8,2), (7,2), (7,3), (7,4) dan (8,4) telah diproses. Titik (9,4), (9,3) dan (9,2) belum diproses dan masih dalam batas sehingga diberi warna merah.



Gambar 4.42 Langkah 6

Langkah 7:

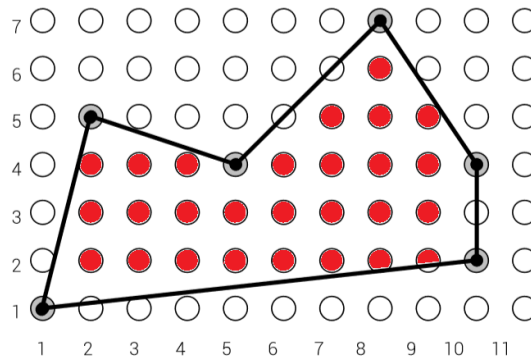
Kemudian diambil titik (8,4) sebagai titik tengah dan titik tetangganya yaitu (7,4), (7,5), (8,5), (9,5), (9,4), (9,3), (8,3), dan (7,3). Titik (9,4), (9,3), (8,3), (7,3) dan (7,4) telah diproses. Titik (7,5), (8,5) dan (9,5) belum diproses dan masih dalam batas sehingga diberi warna merah.



Gambar 4.43 Langkah 7

Langkah 8:

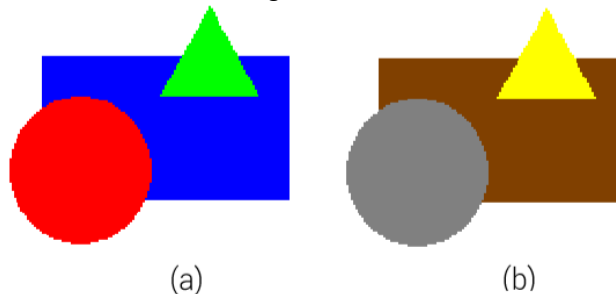
Kemudian diambil titik (8,5) sebagai titik tengah dan titik tetangganya yaitu (7,5), (7,6), (8,6), (9,6), (9,5), (9,4), (8,4), dan (7,4). Titik (9,5), (9,4), (8,4), (7,4) dan (7,5) telah diproses. Titik (8,6) belum diproses dan masih dalam batas sehingga diberi warna merah, sedangkan titik (7,6) dan (9,6) berada di luar batas sehingga tidak diwarnai. Inilah hasil akhir dari pewarnaan dengan menggunakan metode 8-connected.



Gambar 3.44 Langkah 8

### 3.2.3 Flood-Fill Algorithm

Terkadang kita ingin mewarnai (atau memberi warna yang baru) pada sebuah area yang mempunyai warna lebih dari satu. Perhatikan gambar 3.45 berikut:



Gambar 3.45 Penggantian warna obyek menggunakan Flood-Fill Algorithm.

(a) lingkaran berwarna merah, segitiga berwarna hijau dan persegi berwarna biru. Obyek tersebut warnanya diubah menjadi (b) lingkaran berwarna abu-abu, segitiga berwarna kuning dan persegi berwarna coklat

Proses flood fill mirip dengan boundary fill, dimana inti dari proses ini adalah mengubah suatu warna menjadi warna yang lain. Algoritma ini dimulai dari titik yang berada didalam area (x,y) dan mengganti semua pikselnya dengan warna baru sehingga bagian dalam area mempunyai warna yang sama. Pengujian titik tetangga bisa menggunakan 4-tetangga atau 8-tetangga.

Berikut adalah prosedur algoritma *Flood Fill*:

```

Procedure floodFill (x,y,fillColor,oldColor);
Begin
  If getpixel(x,y)= oldColor then
  Begin
    Putpixel (x,y,fillColor);
    floodFill (x+1,y,fillColor,oldColor);
    floodFill (x-1,y,fillColor,oldColor);
    floodFill (x,y+1,fillColor,oldColor);
    floodFill (x,y-1,fillColor,oldColor);
  End;
End;

```

Tabel 3.1 Perbandingan Boundary-Fill Algorithm dan Flood-Fill Algorithm

| Dasar perbandingan     | Algoritma Flood-Fill                                                                                   | Algoritma Boundary-Fill                                  |
|------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Dasar                  | Dapat memiliki area yang mengandung beberapa warna.                                                    | Mendefinisikan area dengan satu warna.                   |
| Proses melukis         | Warna acak dapat digunakan untuk mewarnai bagian interior kemudian yang lama diganti dengan yang baru. | Poin interior diwarnai dengan terus mencari warna batas. |
| Konsumsi memori        | Tinggi                                                                                                 | Rendah                                                   |
| Kecepatan              | Relatif lebih lambat                                                                                   | Cepat                                                    |
| Kompleksitas algoritma | Sederhana relatif                                                                                      | Rumit                                                    |

Perbedaan Kunci Antara Algoritma *Flood-fill* dan *Boundary-fill*:

1. Algoritma Flood-fill dapat memproses gambar yang dilukis oleh lebih dari satu warna batas. Di sisi lain, algoritma Boundary-fill hanya dapat berfungsi gambar yang dibuat oleh warna batas tunggal.
2. Pewarnaan dalam Flood-fill dilakukan hanya dengan mengganti setiap titik warna dengan warna yang ditunjuk. Sebaliknya, algoritma Boundary-fill memeriksa warna batas pada saat melukis suatu daerah.
3. Persyaratan memori dalam Flood-fill lebih tinggi karena tidak menjamin jumlah sub-fill yang dihasilkan. Sebaliknya, Boundary-fill membutuhkan jumlah memori yang lebih sedikit.
4. Algoritma Flood-fill lebih cepat dan rumit daripada algoritme Boundary-fill.

### 3.3 Implementasi Algoritma Filling Polygon

#### 3.3.1 Scan-Line Fill Algorithm

Kode lengkap untuk Algoritma Area Filling Scan Line Objek Poligon dengan Java:

```
import java.util.Collections;
int[][] polygon = {
    {100, 100},
    {200, 80},
    {250, 150},
    {180, 200},
    {120, 180}
};

void setup() {
    size(400, 300);
    background(255);
    stroke(0);
    fill(200);

    // Draw outline of polygon
    beginShape();
    for (int i = 0; i < polygon.length; i++) {
        vertex(polygon[i][0], polygon[i][1]);
    }
    endShape(CLOSE);
}
```

```

    // Apply scan-line fill
    scanlineFill(polygon);
}

void scanlineFill(int[][] poly) {
    int ymin = poly[0][1];
    int ymax = poly[0][1];

    // Find ymin and ymax
    for (int i = 1; i < poly.length; i++) {
        if (poly[i][1] < ymin) ymin = poly[i][1];
        if (poly[i][1] > ymax) ymax = poly[i][1];
    }

    // For each scanline
    for (int y = ymin; y <= ymax; y++) {
        ArrayList<Float> interX = new ArrayList<Float>();

        // Find intersections with polygon edges
        for (int i = 0; i < poly.length; i++) {
            int next = (i + 1) % poly.length;
            int x1 = poly[i][0], y1 = poly[i][1];
            int x2 = poly[next][0], y2 = poly[next][1];

            if ((y1 < y && y2 >= y) || (y2 < y && y1 >= y)) {
                float x = x1 + (float)(y - y1) * (x2 - x1) / (y2 - y1);
                interX.add(x);
            }
        }

        // Sort intersection points
        Collections.sort(interX);

        // Fill between pairs of intersection points
        for (int i = 0; i < interX.size() - 1; i += 2) {
            float xStart = interX.get(i);
            float xEnd = interX.get(i + 1);
            line(xStart, y, xEnd, y);
        }
    }
}

```

### 3.3.2 Boundary-Fill algorithm

Kode lengkap untuk Boundary-Fill Algorithm dengan Java:

```

import java.util.Stack;

color boundaryColor = color(0);           // Black outline
color fillColor = color(255, 0, 0);       // Red fill color

void setup() {
    size(500, 500);
}

```

```
background(255);
noSmooth(); // Important to prevent anti-aliasing when working
with pixel-level color

drawPolygon();

// Choose a seed point inside the polygon
int seedX = 250;
int seedY = 250;

// Only fill if the point is not already boundary or filled
if (!colorsSimilar(get(seedX, seedY), boundaryColor) &&
    !colorsSimilar(get(seedX, seedY), fillColor)) {
    boundaryFill(seedX, seedY);
}
}

void drawPolygon() {
    stroke(boundaryColor);
    fill(255); // Optional: white interior before fill
    beginShape();
    vertex(150, 150);
    vertex(350, 150);
    vertex(400, 300);
    vertex(250, 400);
    vertex(100, 300);
    endShape(CLOSE);
}

void boundaryFill(int x, int y) {
    Stack<PVector> stack = new Stack<PVector>();
    stack.push(new PVector(x, y));

    while (!stack.isEmpty()) {
        PVector point = stack.pop();
        int px = int(point.x);
        int py = int(point.y);

        if (px < 0 || px >= width || py < 0 || py >= height) continue;

        color current = get(px, py);
        if (!colorsSimilar(current, boundaryColor) &&
            !colorsSimilar(current, fillColor)) {
            set(px, py, fillColor);

            // Push neighboring pixels (4-connected)
            stack.push(new PVector(px + 1, py));
            stack.push(new PVector(px - 1, py));
            stack.push(new PVector(px, py + 1));
            stack.push(new PVector(px, py - 1));
        }
    }
}
```

```

    updatePixels(); // Apply changes to the screen
}

// Helper function to compare colors with tolerance
boolean colorsSimilar(color c1, color c2) {
    float tolerance = 5;
    return abs(red(c1) - red(c2)) < tolerance &&
           abs(green(c1) - green(c2)) < tolerance &&
           abs(blue(c1) - blue(c2)) < tolerance;
}

```

### 3.3.3 Flood-Fill Algorithm

Kode lengkap untuk Flood-Fill Algorithm dengan Java:

```

import java.util.Stack;

color fillColor = color(0, 255, 0); // Green fill color
color backgroundColor = color(255); // White background

void setup() {
    size(500, 500);
    background(backgroundColor);
    noSmooth(); // Important to disable anti-aliasing

    drawPolygon();

    int seedX = 250;
    int seedY = 250;

    // Only fill if the seed point is not already filled
    if (colorsSimilar(get(seedX, seedY), backgroundColor)) {
        floodFill(seedX, seedY);
    }
}

void drawPolygon() {
    stroke(0); // Black border
    fill(255); // White fill (backgroundColor)
    beginShape();
    vertex(150, 150);
    vertex(350, 150);
    vertex(400, 300);
    vertex(250, 400);
    vertex(100, 300);
    endShape(CLOSE);
}

void floodFill(int x, int y) {
    Stack<PVector> stack = new Stack<PVector>();
    stack.push(new PVector(x, y));

    while (!stack.isEmpty()) {

```



```

PVector point = stack.pop();
int px = int(point.x);
int py = int(point.y);

if (px < 0 || px >= width || py < 0 || py >= height) continue;

color current = get(px, py);
if (colorsSimilar(current, backgroundColor)) {
    set(px, py, fillColor);

    // 4-connected neighbors
    stack.push(new PVector(px + 1, py));
    stack.push(new PVector(px - 1, py));
    stack.push(new PVector(px, py + 1));
    stack.push(new PVector(px, py - 1));
}
}

updatePixels();
}

// Helper to compare colors with tolerance
boolean colorsSimilar(color c1, color c2) {
    float tolerance = 5;
    return abs(red(c1) - red(c2)) < tolerance &&
           abs(green(c1) - green(c2)) < tolerance &&
           abs(blue(c1) - blue(c2)) < tolerance;
}

```

### 3.4 Soal Latihan

1. Diketahui : polygon = {(2,1), (3,6), (5,4), (8,8), (10,4), (12,2), (2,1)}, lakukan Area Filling menggunakan:
  - a. Algoritma Scan-Line Fill
  - b. Boundary Fill

### Daftar Pustaka

1. Andono, Pulung Nurtantio., Sutojo, T. 2016. Konsep Grafika Komputer. Yogyakarta: CV Andi Offset.
2. David F. Rogers, Alan J. Adams, Mathematical Elements for Computer Graphics (2nd edition), McGraw-Hill, 1989.
3. Ferguson, Ian. Computer Graphics Via Java. Ab-Libris, 2002.
4. Herriyance, Hendryan Winata, Analisa Algoritma Boundary Fill 4-Connected dan 8-Connected pada Image Dua Dimensi (2D), Jurnal SAINTIKOM Vol. 16, No. 1, Januari 2017.
5. John F. Hughes, Andries Van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, Kurt Akeley, Computer Graphics: Principles and Practice (3rd edition), Addison-Wesley, 2014.
6. Klawonn, Frank. 2012. *Introduction to Computer Graphics Using Java 2D and 3D 2<sup>nd</sup> Edition*. Springer-Verlag: London.